TEAMBOOK
PROGRAMACIÓN COMPETITIVA

Caja negra de algoritmos, estructuras de datos y matemáticas

Basado en el árbol completo de `stevenhalim/cpbook-code` (CP4)

Implementaciones consolidadas en GNU++17

Este documento está diseñado para resolver problemas bajo presión: identificar el patrón, consultar la implementación, adaptar índices y tipos, y verificar las condiciones de uso. No reemplaza el razonamiento ni la lectura del enunciado.

Versión de trabajo: noviembre de 2026

Cómo usar esta caja negra

El repositorio de referencia contiene código de CP4 en C++17, Java 11, Python 3 y OCaml. Aquí se reúnen los algoritmos y patrones que aparecen en sus nueve capítulos y se normalizan a GNU++17 para que el equipo use una sola sintaxis. Los programas concretos de UVa/Kattis se tratan como ejemplos de aplicación; no se copian soluciones completas de cada juez, sino que se extrae la técnica reutilizable.

Protocolo de uso en concurso:

1. Clasificar el problema por su señal: consultas de rango, conectividad, camino mínimo, subsecuencia, divisibilidad, coincidencia de patrones, etc.
2. Confirmar restricciones: N, M, pesos negativos, necesidad de actualizar, memoria y precisión.
3. Elegir el algoritmo correcto usando la tabla de decisión de cada capítulo.
4. Copiar la plantilla, adaptar tipos e índices y escribir una prueba mínima.
5. Verificar casos límite: vacío, uno, duplicados, desconectado, solución inexistente, overflow y empates.

Ojo. Una plantilla no es una solución automática. Antes de pegarla, comprueba las precondiciones escritas en su sección. En particular: Dijkstra no admite aristas negativas; la Sparse Table de esta versión es para operaciones idempotentes; el algoritmo de Kruskal calcula un bosque si el grafo no es conexo.

Notación: V vértices, E aristas, N elementos, M módulo, s origen, t destino, n longitud, log logaritmo en base 2. Los índices son 0-based salvo que se indique expresamente lo contrario.

Índice de consulta rápida

Si el problema pide...	Ir a...
leer hasta EOF/centinela, parsing o simular formato	Cap. 1
ordenar, contar frecuencias, buscar k-ésimo	Cap. 2
conectividad de conjuntos / Kruskal	DSU, Cap. 2 y 4
suma de prefijo / rango con actualizaciones	Fenwick, Cap. 2
consulta de rango general / lazy	Segment Tree, Cap. 2
subconjuntos, permutaciones, restricciones pequeñas	Cap. 3 y 8
mínimo factible / máximo factible	búsqueda binaria, Cap. 3
subarreglo máximo, subsecuencia, mochila, intervalos	DP, Cap. 3
camino no ponderado / pesos positivos / pesos negativos	BFS / Dijkstra / Bellman-Ford, Cap. 4
conectar todos al menor costo	MST, Cap. 4
primos, divisores, inversos, combinaciones y módulo	Cap. 5
buscar patrón en texto	KMP / Rolling Hash, Cap. 6
prefijos de palabras	Trie, Cap. 6
subcadenas repetidas / sufijos	Suffix Array + LCP, Cap. 6
rectas, círculos, triángulos, polígonos	Cap. 7
flujo máximo / asignación	Cap. 4 (matching) y Cap. 8 (flujo)
LCA, HLD, Sparse Table, Gauss, Pollard Rho, MCMF	Cap. 9

Índice general

Cómo usar esta caja negra **i**

Índice de consulta rápida I

1. Fundamentos, entrada/salida y problemas Ad Hoc **1**

1.1. Plantilla base GNU++17 1

1.2. Patrones de lectura 1

1.2.1. Cantidad conocida de casos 1

1.2.2. Centinela 1

1.2.3. Hasta EOF 1

1.2.4. Líneas completas y tokens dentro de cada línea 2

1.3. Cadenas y caracteres 2

1.3.1. Construir frecuencia y palabras 2

1.3.2. Concatenar líneas partidas por guion 2

1.4. Aritmética cotidiana y conversiones 2

1.4.1. Conversión de base b a decimal 3

1.4.2. Números romanos 3

1.5. Problemas Ad Hoc: patrones que no necesitan una estructura pesada 3

1.6. Checklist antes de enviar 3

2. Estructuras de datos y biblioteca estándar **4**

2.1. Complejidad de referencia 4

2.2. Contenedores lineales 4

2.3. Ordenamiento y búsqueda binaria 4

2.3.1. Orden natural y descendente 4

2.3.2. Ordenamiento por varios criterios 5

2.3.3. lower_bound y upper_bound 5

2.3.4. Permutaciones 5

2.4. Mapas, conjuntos y heaps 5

2.5. Quickselect / k-ésimo elemento 5

2.6. Manipulación de bits y bitmask 6

2.7. Union-Find Disjoint Sets (DSU / UFDS) 6

2.8. Fenwick Tree / Binary Indexed Tree 6

2.9. Segment Tree y Lazy Propagation 7

2.10. Sparse Table (tabla dispersa) 8

2.11. Otras estructuras del repositorio 8

3. Paradigmas de resolución de problemas **9**

3.1. Complete Search / fuerza bruta 9

3.2. Backtracking . . . 9
3.2.1. N-Reinas con máscaras . . . 10
3.3. Divide and Conquer . . . 10
3.3.1. Búsqueda binaria sobre respuesta . . . 10
3.3.2. Búsqueda ternaria . . . 10
3.4. Greedy / algoritmos voraces . . . 10
3.5. Programación dinámica (DP) . . . 11
3.5.1. Memoización top-down . . . 11
3.5.2. Tabulación bottom-up . . . 11
3.5.3. Máxima suma de subarreglo (Kadane) . . . 11
3.5.4. LIS: subsecuencia creciente más larga . . . 12
3.5.5. Mochila 0/1 . . . 12
3.5.6. Cambio de monedas . . . 12
3.5.7. DP de intervalos: Cutting Sticks . . . 12
3.5.8. DP sobre árboles: vertex cover mínimo . . . 12
3.5.9. DP de conteo: sumar S con K términos no negativos . . . 12
3.5.10. DP de presupuesto / compras . . . 13
3.5.11. DP de recolección por alturas (ACORN) . . . 13
3.5.12. TSP pequeño / Beepers . . . 13
3.5.13. DP con bitmask / matching perfecto pequeño . . . 13
3.6. Reconocer el paradigma correcto . . . 13

4. Grafos **14**
4.1. Representaciones . . . 14
4.2. BFS: recorrido y camino más corto sin pesos . . . 14
4.2.1. BFS para bipartición . . . 14
4.3. DFS, componentes y flood fill . . . 14
4.4. Topological Sort . . . 15
4.5. Componentes fuertemente conexas (SCC) . . . 15
4.6. Puentes y puntos de articulación . . . 16
4.7. Caminos mínimos desde una fuente . . . 16
4.7.1. Dijkstra . . . 16
4.7.2. Bellman-Ford . . . 16
4.7.3. Floyd-Warshall: todos contra todos . . . 17
4.8. Árbol de expansión mínima (MST) . . . 17
4.8.1. Kruskal . . . 17
4.8.2. Prim . . . 17
4.9. Euler: camino y circuito . . . 18
4.10. Matching bipartito máximo . . . 18
4.11. Elección del algoritmo de grafos . . . 19

5. Matemáticas para programación competitiva **20**
5.1. Aritmética entera básica . . . 20
5.1.1. GCD, LCM y Euclides extendido . . . 20
5.2. Aritmética modular . . . 20
5.2.1. Exponenciación binaria . . . 20
5.2.2. Inverso modular . . . 21
5.3. Primos y factorización . . . 21

5.3.1. Criba de Eratóstenes . . . 21
5.3.2. Factorización por división de prueba . . . 21
5.3.3. Funciones de divisores . . . 21
5.3.4. Legendre: exponente de primo en $n!$ . . . 22
5.4. Combinatoria . . . 22
5.4.1. Fórmulas esenciales . . . 22
5.4.2. Factoriales e inversos precomputados (módulo primo) . . . 22
5.4.3. Catalan y Fibonacci . . . 23
5.5. Teorema chino del resto (CRT) . . . 23
5.6. Otros patrones matemáticos del repositorio . . . 23

6. Procesamiento de cadenas **24**
6.1. Coincidencia de patrones . . . 24
6.1.1. Naive matching . . . 24
6.1.2. KMP: Knuth-Morris-Pratt . . . 24
6.1.3. Rabin-Karp / rolling hash . . . 24
6.2. Trie (árbol de prefijos) . . . 25
6.3. Alineamiento y distancia de edición . . . 25
6.4. Suffix Array y LCP . . . 26

7. Geometría computacional **27**
7.1. Puntos y vectores . . . 27
7.2. Orientación y área . . . 27
7.2.1. Fórmula del zapatero (shoelace) . . . 27
7.3. Rectas . . . 27
7.4. Proyección y distancia punto-segmento . . . 28
7.5. Segmentos . . . 28
7.6. Círculos . . . 28
7.7. Triángulos . . . 28
7.8. Polígonos . . . 29
7.8.1. Punto dentro/fuera . . . 29
7.8.2. Convexidad . . . 29
7.8.3. Corte de polígono . . . 29
7.9. Errores geométricos frecuentes . . . 29

8. Búsqueda avanzada y flujo máximo **30**
8.1. Búsqueda en espacio de estados . . . 30
8.1.1. BFS de estados . . . 30
8.1.2. Bidirectional BFS / Meet in the Middle . . . 30
8.2. IDA*: Iterative Deepening A* . . . 30
8.3. DP y backtracking con máscaras . . . 31
8.4. Flujo máximo . . . 31
8.4.1. Dinic . . . 31
8.5. Corte mínimo . . . 32

9. Temas avanzados del repositorio **33**
9.1. LCA: Lowest Common Ancestor . . . 33
9.1.1. Binary lifting . . . 33

9.1.2. Euler Tour + RMQ . . . 33
9.2. Heavy-Light Decomposition (HLD) . . . 34
9.3. Sparse Table avanzada . . . 34
9.4. Eliminación Gaussiana . . . 34
9.5. Miller-Rabin y Pollard Rho (factorización 64-bit) . . . 34
9.6. Minimum-Cost Maximum-Flow (MCMF) . . . 35
9.7. Teorema chino del resto: aplicación astronómica . . . 36
9.8. Resumen de algoritmos avanzados . . . 36

A. Mapa de cobertura del repositorio original **37**
A.1. Correspondencia por capítulo . . . 37
A.2. Archivos de apoyo y ejemplos de jueces . . . 38
A.3. Qué se consolidó y qué se adaptó . . . 38
A.4. Fuentes y atribución . . . 38

Recordatorio final para el equipo **39**

CAPÍTULO 1

Fundamentos, entrada/salida y problemas Ad Hoc

1.1 Plantilla base GNU++17

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;
using pii = pair<int,int>;
using vi = vector<int>;
const int INF = 1'000'000'000;
const ll LINF = (1LL<<62);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // leer, procesar, imprimir
    return 0;
}
```

Regla de E/S: usa `cin/cout` con sincronización desactivada o `scanf/printf`; no mezcles ambos estilos sin necesidad. `endl` fuerza un flush: prefiere `'\n'`.

1.2 Patrones de lectura

1.2.1 Cantidad conocida de casos

```cpp
int tc; cin >> tc;
while (tc--) {
    int a, b; cin >> a >> b;
    cout << a+b << '\n';
}
```

1.2.2 Centinela

```cpp
int n;
while (cin >> n && n != 0) {
    // procesar caso
}
```

1.2.3 Hasta EOF

```cpp
int a, b;
while (cin >> a >> b) {
    cout << a+b << '\n';
}
```

1.2.4 Líneas completas y tokens dentro de cada línea

```cpp
string line;
while (getline(cin, line)) {
    stringstream ss(line);
    string token;
    while (ss >> token) {
        // procesar token
    }
}
```

Ojo. Después de leer con `cin » x`, el salto pendiente puede hacer que el siguiente `getline` lea una línea vacía. Usa `cin.ignore(..., '\n')` antes de pasar a leer líneas completas.

1.3 Cadenas y caracteres

Operación	C++17
Longitud / acceso	`s.size()`, `s[i]`
Subcadena	`s.substr(pos, len)`
Buscar texto	`s.find(pat)`; comparar con `string::npos`
Insertar / borrar	`s.insert(pos, x)`, `s.erase(pos, len)`
Ordenar caracteres	`sort(s.begin(), s.end())`
Convertir mayúscula/minúscula	`toupper((unsigned char)c)`, `tolower((unsigned char)c)`
Clasificar	`isdigit`, `isalpha`, `isspace`
Dividir tokens	`stringstream`

```cpp
string s; getline(cin, s);
int digits=0, letters=0, vowels=0;
for (unsigned char ch : s) {
    digits += isdigit(ch);
    letters += isalpha(ch);
    char c = (char)tolower(ch);
    vowels += (c=='a'||c=='e'||c=='i'||c=='o'||c=='u');
}
```

1.3.1 Construir frecuencia y palabras

```cpp
map<string,int> freq;
stringstream ss(text);
string word;
while (ss >> word) ++freq[word];
```

1.3.2 Concatenar líneas partidas por guion

En algunos formatos editoriales, un guion al final de línea indica que la palabra continúa sin espacio. Construye el texto con una variable `continua`: añade espacio entre líneas solo si la línea anterior no terminó en guion; elimina el guion antes de concatenar. No elimines guiones internos.

1.4 Aritmética cotidiana y conversiones

```cpp
// Suma segura: promocionar ANTES de multiplicar
ll producto = 1LL * a * b;

// División de enteros hacia abajo (floor) para b > 0
ll floor_div(ll a, ll b) {
    ll q = a / b, r = a % b;
    if (r != 0 && a < 0) --q;
    return q;
}

// Redondeo de un double positivo al entero más cercano
ll redondear(double x) { return llround(x); }
```

1.4.1 Conversión de base b a decimal

Si los dígitos son $d_k \dots d_1 d_0$, su valor es $\sum_{i=0}^{k} d_i b^i$. Para convertir una cadena a base decimal se puede acumular: `ans = ans*b + digit`. Para convertir un entero a base b, extrae restos y revierte el resultado. Usa `long long` o una librería de enteros grandes si el resultado excede 64 bits.

1.4.2 Números romanos

Tabla de símbolos: I=1, V=5, X=10, L=50, C=100, D=500, M=1000. Para leer, si un símbolo es menor que el siguiente se resta; en otro caso se suma. Para generar, usa los bloques greedy `1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1`. Revisa las reglas de rango y representación exigidas por el juez.

1.5 Problemas Ad Hoc: patrones que no necesitan una estructura pesada

- Simulación literal: mantener estado y aplicar operaciones en el orden exacto.
- Conteo: frecuencias con arreglo, `map` o `unordered_map`.
- Validación por condiciones: traducir cada frase del enunciado a un booleano explícito.
- Fechas/horas: convertir a una unidad común (segundos/minutos) antes de comparar.
- Búsqueda de patrón dentro de una cadena corta: `find` o `strstr`; para textos grandes usa KMP.
- Permutación o formato especial: separar parsing, transformación y salida; respeta espacios y líneas en blanco.

1.6 Checklist antes de enviar

- ¿El juez tiene varios casos, centinela o EOF?
- ¿Hay que imprimir una línea vacía entre casos?
- ¿El formato requiere ceros a la izquierda o decimales fijos?
- ¿Hay líneas que contienen espacios?
- ¿Se debe conservar mayúsculas, puntuación o espacios?
- ¿Hay riesgo de que una operación intermedia desborde aunque el resultado final quepa?

CAPÍTULO 2

Estructuras de datos y biblioteca estándar

2.1 Complejidad de referencia

Estructura / operación	Tiempo típico	Uso
Arreglo / `vector` acceso	$\mathcal{O}(1)$	índice conocido
`vector` insertar al final	amortizado $\mathcal{O}(1)$	lista dinámica
Insertar/borrar en medio de vector	$\mathcal{O}(N)$	desplaza elementos
`stack`, `queue`	$\mathcal{O}(1)$	LIFO / FIFO
`deque` extremos	$\mathcal{O}(1)$	doble extremo
`set`/`map`	$\mathcal{O}(\log N)$	orden, búsqueda
`unordered_set`/map	promedio $\mathcal{O}(1)$	hashing
`priority_queue`	top $\mathcal{O}(1)$; push/pop $\mathcal{O}(\log N)$	extraer prioridad
Fenwick	update/query $\mathcal{O}(\log N)$	suma prefijo dinámica
Segment Tree	query/update $\mathcal{O}(\log N)$	rango general
Sparse Table	query $\mathcal{O}(1)$, preproceso $\mathcal{O}(N \log N)$	rango estático idempotente
DSU	casi $\mathcal{O}(1)$ amortizado	componentes dinámicas

2.2 Contenedores lineales

```
vector<int> v;
v.push_back(x); v.pop_back();
int n = (int)v.size();
sort(v.begin(), v.end());

stack<int> st; st.push(x); int a=st.top(); st.pop();
queue<int> q; q.push(x); int b=q.front(); q.pop();
deque<int> dq; dq.push_front(x); dq.push_back(y);
```

Vector redimensionable: `reserve(k)` reserva capacidad sin cambiar tamaño; `resize(k)` cambia el tamaño. No accedas a `v[i]` cuando $i \geq v.size()$.

2.3 Ordenamiento y búsqueda binaria

2.3.1 Orden natural y descendente

```
sort(a.begin(), a.end());
sort(a.rbegin(), a.rend());
reverse(a.begin(), a.end());
```

2.3.2 Ordenamiento por varios criterios

```cpp
struct Team { int id, solved, penalty; };
bool cmp(const Team& a, const Team& b) {
    if (a.solved != b.solved) return a.solved > b.solved;
    if (a.penalty != b.penalty) return a.penalty < b.penalty;
    return a.id < b.id;
}
// Alternativa compacta: sort(vector<tuple<...>>)
```

Ojo. Un comparador de `sort` debe definir un orden estricto débil. No uses `<=`; si hay igualdad, devuelve `false`. En flotantes, evita comparadores inconsistentes basados en EPS si pueden violar transitividad.

2.3.3 lower_bound y upper_bound

```cpp
auto lo = lower_bound(v.begin(), v.end(), x); // primer >= x
auto hi = upper_bound(v.begin(), v.end(), x); // primer > x
int count_equal = hi - lo;
bool exists = (lo != v.end() && *lo == x);
```

2.3.4 Permutaciones

```cpp
sort(v.begin(), v.end());
do {
    // procesar permutación actual
} while (next_permutation(v.begin(), v.end()));
```

Complejidad: Ordenamiento $\mathcal{O}(N \log N)$; lower/upper bound $\mathcal{O}(\log N)$; next_permutation $\mathcal{O}(N)$ por avance.

2.4 Mapas, conjuntos y heaps

```cpp
map<string,int> mp;
mp[key]++;                          // inserta si no existe
if (mp.count(key)) { /* existe */ }
auto it = mp.lower_bound(key);      // primer key >= buscado

set<int> s;
s.insert(x); s.erase(x);
auto it2 = s.find(x);

unordered_map<long long,int> cnt;
priority_queue<int> maxHeap;
priority_queue<int, vector<int>, greater<int>> minHeap;
```

Diferencias: `map/set` mantienen orden y garantizan $\mathcal{O}(\log N)$; `unordered_*` no ordenan y ofrecen $\mathcal{O}(1)$ promedio, pero pueden degradarse; `priority_queue` solo expone el extremo prioritario.

2.5 Quickselect / k-ésimo elemento

Para obtener el elemento de índice k (0-based) sin ordenar todo:

```cpp
// promedio lineal; modifica el vector
nth_element(v.begin(), v.begin()+k, v.end());
int kth = v[k];
```

Complejidad: Promedio $\mathcal{O}(N)$; peor caso teórico $\mathcal{O}(N^2)$ en implementaciones típicas de selección aleatoria.

Para varios rangos ordenados, un `sort` completo suele ser más conveniente. Recuerda que el k-ésimo menor en la convención 1-based usa índice `k-1`.

2.6 Manipulación de bits y bitmask

Para representar subconjuntos de $\{0, \ldots, n-1\}$ con un entero, el bit i indica pertenencia.

```cpp
// Preferir unsigned cuando se manipulan bits
using ull = unsigned long long;
bool on = (mask >> i) & 1ULL;
mask |= (1ULL << i);          // activar i
mask &= ~(1ULL << i);         // apagar i
mask ^= (1ULL << i);          // alternar i
int cnt = __builtin_popcountll(mask);
int i = __builtin_ctzll(mask); // mask != 0
ull lowbit = mask & -mask;
bool power2 = mask && !(mask & (mask-1));
for (ull s=mask; s; s=(s-1)&mask) {
    // recorrer submáscaras no vacías
}
```

Expresión	Significado
$S \,\&\, (1 \ll j)$	¿está activo el bit j?
$S \mid (1 \ll j)$	activar bit j
$S \,\&\, \sim(1 \ll j)$	desactivar bit j
$S \oplus (1 \ll j)$	alternar bit j
$S \,\&\, (-S)$	bit activo menos significativo
$S \,\&\, (S-1)$	apagar el bit activo menos significativo
$(S \,\&\, (N-1))$	equivale a S mód N si N es potencia de 2

Ojo. No hagas `1 « i` si i puede acercarse al ancho de `int`. Usa `1LL « i` o `1ULL « i`, y comprueba que el desplazamiento sea menor que 64. `ctz(0)` no está definido.

2.7 Union-Find Disjoint Sets (DSU / UFDS)

Señal del problema: Conectar pares, preguntar si dos vértices están conectados, contar componentes, Kruskal.
Herramienta: DSU con compresión de caminos y unión por tamaño.

```cpp
struct DSU {
    vector<int> p, sz;
    int comps;
    DSU(int n=0) { init(n); }
    void init(int n) {
        p.resize(n); iota(p.begin(), p.end(), 0);
        sz.assign(n,1); comps=n;
    }
    int find(int x) {
        return p[x]==x ? x : p[x]=find(p[x]);
    }
    bool unite(int a,int b) {
        a=find(a); b=find(b);
        if(a==b) return false;
        if(sz[a]<sz[b]) swap(a,b);
        p[b]=a; sz[a]+=sz[b]; --comps;
        return true;
    }
    bool same(int a,int b) { return find(a)==find(b); }
    int size(int x) { return sz[find(x)]; }
};
```

Complejidad: Amortizado $\mathcal{O}(\alpha(N))$ por operación, prácticamente constante.

Ojo. DSU no admite deshacer uniones de forma directa ni responder conectividad histórica arbitraria sin técnicas adicionales.

2.8 Fenwick Tree / Binary Indexed Tree

Señal del problema: Suma de prefijo/rango con actualizaciones puntuales; frecuencias dinámicas; inversión de prefijos.

Herramienta: Fenwick, índices 1-based.

```
struct Fenwick {
    int n; vector<long long> bit;
    Fenwick(int n=0): n(n), bit(n+1,0) {}
    void add(int i, long long v) {
        for (; i<=n; i+=i&-i) bit[i]+=v;
    }
    long long sumPrefix(int i) const {
        long long s=0;
        for (; i>0; i-=i&-i) s+=bit[i];
        return s;
    }
    long long sumRange(int l,int r) const {
        return l>r ? 0 : sumPrefix(r)-sumPrefix(l-1);
    }
};
```

Complejidad: Construcción simple $\mathcal{O}(N \log N)$; suma y actualización $\mathcal{O}(\log N)$; existe construcción lineal.

Actualización de rango + consulta puntual: aplica diferencias: para sumar v en $[l, r]$, ejecuta `add(l,+v)` y `add(r+1,-v)`; el valor del punto i es `sumPrefix(i)`.

Actualización de rango + suma de rango: usa dos Fenwick, B_1, B_2. Para añadir v a $[l, r]$: en B_1 añade $+v$ en l, $-v$ en $r+1$; en B_2 añade $v(l-1)$ en l y $-vr$ en $r+1$. Entonces $prefix(x) = x \cdot sum(B_1, x) - sum(B_2, x)$. La suma $[l, r]$ se obtiene por diferencia de prefijos.

2.9 Segment Tree y Lazy Propagation

Señal del problema: Consultas de rango con actualizaciones; mínimo, máximo, suma, gcd u otra operación asociativa.
Herramienta: Segment Tree; lazy propagation si la actualización afecta un intervalo.

```
struct SegTree {
    int n; vector<long long> st, lazy;
    SegTree(const vector<long long>& a) {
        n=(int)a.size(); st.assign(4*n,0); lazy.assign(4*n,0);
        build(1,0,n-1,a);
    }
    void build(int p,int l,int r,const vector<long long>& a) {
        if(l==r) { st[p]=a[l]; return; }
        int m=(l+r)/2;
        build(p*2,l,m,a); build(p*2+1,m+1,r,a);
        st[p]=st[p*2]+st[p*2+1];
    }
    void apply(int p,int l,int r,long long v) {
        st[p]+=v*(r-l+1); lazy[p]+=v;
    }
    void push(int p,int l,int r) {
        if(lazy[p]==0 || l==r) return;
        int m=(l+r)/2;
        apply(p*2,l,m,lazy[p]);
        apply(p*2+1,m+1,r,lazy[p]);
        lazy[p]=0;
    }
    void add(int p,int l,int r,int ql,int qr,long long v) {
        if(qr<l || r<ql) return;
        if(ql<=l && r<=qr) { apply(p,l,r,v); return; }
        push(p,l,r); int m=(l+r)/2;
        add(p*2,l,m,ql,qr,v); add(p*2+1,m+1,r,ql,qr,v);
        st[p]=st[p*2]+st[p*2+1];
    }
    long long query(int p,int l,int r,int ql,int qr) {
        if(qr<l || r<ql) return 0;
        if(ql<=l && r<=qr) return st[p];
        push(p,l,r); int m=(l+r)/2;
        return query(p*2,l,m,ql,qr)+query(p*2+1,m+1,r,ql,qr);
    }
    void add(int l,int r,long long v) { add(1,0,n-1,l,r,v); }
    long long query(int l,int r) { return query(1,0,n-1,l,r); }
};
```

Complejidad: Build $\mathcal{O}(N)$; consulta y actualización $\mathcal{O}(\log N)$.

Ojo. Esta implementación es para suma de rango y actualización de rango por suma. Para asignación de rango, máximo/mínimo, gcd u otra operación, cambia la combinación y la semántica lazy. El valor neutro de consulta también cambia.

2.10 Sparse Table (tabla dispersa)

Señal del problema: Arreglo estático, muchas consultas RMQ mínimo/máximo/gcd y ninguna actualización.
Herramienta: Precalcular intervalos de longitudes potencia de 2.

```cpp
struct SparseMin {
    int n, K; vector<int> lg;
    vector<vector<int>> st;
    SparseMin(const vector<int>& a) {
        n=a.size(); K=32-__builtin_clz(n);
        lg.assign(n+1,0);
        for(int i=2;i<=n;i++) lg[i]=lg[i/2]+1;
        st.assign(K, vector<int>(n)); st[0]=a;
        for(int k=1;k<K;k++)
            for(int i=0;i+(1<<k)<=n;i++)
                st[k][i]=min(st[k-1][i], st[k-1][i+(1<<(k-1))]);
    }
    int query(int l,int r) { // inclusive, 0-based
        int k=lg[r-l+1];
        return min(st[k][l], st[k][r-(1<<k)+1]);
    }
};
```

Complejidad: Preproceso $\mathcal{O}(N \log N)$, consulta $\mathcal{O}(1)$, memoria $\mathcal{O}(N \log N)$.

Ojo. La consulta superpone dos bloques; esto funciona con operaciones idempotentes como min, max y gcd, no con suma. Para suma usa prefijos o una estructura que no superponga bloques.

2.11 Otras estructuras del repositorio

- **Lista enlazada:** inserción/borrado $\mathcal{O}(1)$ si ya tienes el iterador; acceso por posición $\mathcal{O}(N)$. En C++ suele bastar `list`, aunque `vector` es más amigable con caché.
- **Arreglo dinámico:** duplicar capacidad cuando se llena da inserción al final amortizada $\mathcal{O}(1)$.
- **BST / AVL:** árbol de búsqueda ordenado. BST puede degenerar a altura N; AVL mantiene altura $\mathcal{O}(\log N)$ mediante rotaciones. En concursos, `set/map` suelen ser preferibles a escribir AVL.
- **PBDS (GNU extension):** árbol ordenado con `order_of_key(x)` y `find_by_order(k)`. No es C++ estándar y no siempre está disponible en jueces.

```cpp
// GNU PBDS: compilar con g++ (no portable a todos los jueces)
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template<class T> using ordered_set = tree<T,null_type,less<T>,
    rb_tree_tag,tree_order_statistics_node_update>;
// order_of_key(x): cuantos elementos son < x
// *find_by_order(k): elemento de indice k (0-based)
```

CAPÍTULO 3

Paradigmas de resolución de problemas

3.1 Complete Search / fuerza bruta

Señal del problema: N pequeño; se pide contar, maximizar o verificar todas las configuraciones.
Herramienta: Enumerar estados sistemáticamente, sin repetir, con poda cuando sea posible.

Permutaciones: $N!$ estados. Usa `next_permutation`.

```
sort(p.begin(),p.end());
do {
    // evaluar p
} while(next_permutation(p.begin(),p.end()));
```

Combinaciones de tamaño k: recursión con siguiente índice; complejidad $\binom{N}{k}$.

```
void choose(int at,int need, vector<int>& cur,
            const vector<int>& a) {
    if(need==0) { /* usar cur */ return; }
    if((int)a.size()-at<need) return; // poda
    for(int i=at;i<(int)a.size();i++) {
        cur.push_back(a[i]);
        choose(i+1,need-1,cur,a);
        cur.pop_back();
    }
}
```

Subconjuntos: para $N \leq 25$ aprox. (depende del tiempo), recorrer máscaras:

```
for(int mask=0; mask<(1<<n); ++mask) {
    for(int i=0;i<n;i++) if(mask&(1<<i)) {
        // elemento i pertenece al subconjunto
    }
}
```

Poda: detener una rama si ya supera el mejor costo posible, si viola restricciones, o si la cota optimista no puede superar la mejor respuesta actual. Define una condición de poda demostrablemente segura; podar por intuición puede eliminar la respuesta correcta.

3.2 Backtracking

Señal del problema: Construir una configuración paso a paso; restricciones locales; contar soluciones.
Herramienta: DFS recursivo + marcar/ desmarcar estado.

```
void dfs(int pos) {
    if(pos==N) { ++ways; return; }
    for(int x : candidates(pos)) {
        if(!valid(pos,x)) continue;
        apply(pos,x);
        dfs(pos+1);
        undo(pos,x);
    }
}
```

3.2.1 N-Reinas con máscaras

```
long long queens(int row,int n,int cols,int dl,int dr,
                 const vector<int>& blocked) {
    if(row==n) return 1;
    int all=(1<<n)-1;
    int free=all & ~(cols|dl|dr|blocked[row]);
    long long ans=0;
    while(free) {
        int bit=free & -free; free-=bit;
        ans+=queens(row+1,n,cols|bit,
                    ((dl|bit)<<1)&all,(dr|bit)>>1,blocked);
    }
    return ans;
}
```

Complejidad: Exponencial en el peor caso; las máscaras permiten operaciones por palabra y una poda fuerte.

3.3 Divide and Conquer

Dividir el problema, resolver subproblemas y combinar. Ejemplos clásicos: merge sort, quicksort, búsqueda binaria y búsqueda ternaria.

3.3.1 Búsqueda binaria sobre respuesta

Señal del problema: Preguntan mínimo/máximo valor factible y la factibilidad es monótona.
Herramienta: Búsqueda binaria de primer verdadero o último verdadero.

```
// pred(x) = true desde cierto punto en adelante
long long lo=0, hi=1'000'000'000LL;
while(lo<hi) {
    long long mid=lo+(hi-lo)/2;
    if(pred(mid)) hi=mid;
    else lo=mid+1;
}
long long first_true=lo;
```

Ojo. Antes de usarla, demuestra monotonía. Ejemplo: si una capacidad x permite completar una tarea, cualquier capacidad mayor también la permite. Para último verdadero usa un mid superior y actualiza el límite de forma que el ciclo progrese.

3.3.2 Búsqueda ternaria

Señal del problema: Optimizar una función unimodal en un intervalo continuo o discreto pequeño.
Herramienta: Comparar dos puntos a un tercio del intervalo.

```
double lo=L, hi=R;
for(int it=0;it<100;it++) {
    double m1=lo+(hi-lo)/3.0;
    double m2=hi-(hi-lo)/3.0;
    if(f(m1)<f(m2)) hi=m2; // mínimo unimodal
    else lo=m1;
}
double x=(lo+hi)/2;
```

Ojo. Para máximos invierte la comparación. En problemas discretos, suele convenir reducir hasta un intervalo corto y revisar cada entero restante.

3.4 Greedy / algoritmos voraces

Señal del problema: Una decisión local óptima puede extenderse a una solución óptima global.
Herramienta: Ordenar por criterio correcto + seleccionar y demostrar intercambio.

Interval scheduling: ordenar por tiempo de finalización ascendente; seleccionar la siguiente actividad que no se solape. Complejidad $\mathcal{O}(N \log N)$.

```cpp
sort(a.begin(),a.end(),[](auto x,auto y){
    return x.end<y.end;
});
int lastEnd=INT_MIN, ans=0;
for(auto x:a) if(x.start>=lastEnd) {
    ++ans; lastEnd=x.end;
}
```

Cubrir un segmento con intervalos: desde la frontera actual, entre todos los intervalos que empiezan antes/justo en la frontera, elegir el que llega más a la derecha; repetir. Si ninguno avanza, es imposible. Complejidad $\mathcal{O}(N \log N)$ al ordenar más el recorrido.

Asignación de recursos / minimizar el máximo: a menudo se combina greedy con búsqueda binaria en respuesta. Para una capacidad candidata, intenta asignar de forma voraz y verifica si se logra.

Ojo. Greedy no significa "elegir lo que parece mejor". Busca una prueba: intercambio, mantenerse siempre al menos tan bien como una solución óptima parcial, o una propiedad de matroides.

3.5 Programación dinámica (DP)

Cuándo usar DP: subproblemas repetidos + estructura óptima. Define con precisión el estado, la transición, los casos base, el orden y la respuesta.

Patrón	Ejemplo de estado
1D	$dp[i]$ = mejor respuesta hasta/desde i
2D / grid	$dp[i][j]$ = respuesta para prefijos o celda
Intervalos	$dp[l][r]$ = costo óptimo en $[l, r]$
Mochila	$dp[i][w]$ = usando primeros i objetos y capacidad w
Máscaras	$dp[mask]$ = mejor valor tras elegir el subconjunto $mask$
Árbol	$dp[u][estado]$ = respuesta del subárbol de u

3.5.1 Memoización top-down

```cpp
vector<long long> memo(N,-1);
long long solve(int i) {
    if(i==N) return 0;
    long long &ans=memo[i];
    if(ans!=-1) return ans;
    ans=...; // transición
    return ans;
}
```

Ojo. Si la respuesta puede valer -1, usa un arreglo `vis` separado o un sentinel fuera del rango. La recursión profunda puede desbordar la pila.

3.5.2 Tabulación bottom-up

Ordena los estados de modo que las dependencias ya estén calculadas. Suele dar control de memoria y evita la pila recursiva.

3.5.3 Máxima suma de subarreglo (Kadane)

Señal del problema: Subarreglo contiguo con suma máxima.
Herramienta: Mantener suma terminando aquí y máximo global.

```cpp
long long best=a[0], cur=a[0];
for(int i=1;i<n;i++) {
    cur=max<long long>(a[i],cur+a[i]);
    best=max(best,cur);
}
```

Complejidad: $\mathcal{O}(N)$ tiempo, $\mathcal{O}(1)$ memoria. Maneja arreglos todos negativos inicializando con $a[0]$.

3.5.4 LIS: subsecuencia creciente más larga

Señal del problema: Subsecuencia estrictamente creciente, no necesariamente contigua.
Herramienta: Colas de mínimos / lower_bound.

```
vector<int> tail;
for(int x:a) {
    auto it=lower_bound(tail.begin(),tail.end(),x);
    if(it==tail.end()) tail.push_back(x);
    else *it=x;
}
int lis=(int)tail.size();
```

Complejidad: $\mathcal{O}(N \log N)$ tiempo, $\mathcal{O}(N)$ memoria. Para subsecuencia no decreciente se usa `upper_bound`. La longitud es directa; reconstruir la secuencia requiere guardar índices predecesores.

3.5.5 Mochila 0/1

```
// peso[i], valor[i], capacidad W; cada objeto como máximo una vez
vector<long long> dp(W+1,0);
for(int i=0;i<n;i++)
    for(int w=W;w>=peso[i];--w)
        dp[w]=max(dp[w],dp[w-peso[i]]+valor[i]);
long long ans=dp[W]; // máximo con capacidad <= W
```

Ojo. El bucle de capacidad va hacia atrás en 0/1. Si va hacia adelante, el mismo objeto puede reutilizarse (mochila ilimitada).

3.5.6 Cambio de monedas

```
const int INF2=1e9;
vector<int> dp(X+1,INF2); dp[0]=0;
for(int c:coins)
    for(int x=c;x<=X;x++)
        dp[x]=min(dp[x],dp[x-c]+1); // monedas ilimitadas
if(dp[X]==INF2) cout<<"imposible\n";
else cout<<dp[X]<<'\n';
```

3.5.7 DP de intervalos: Cutting Sticks

Si los cortes están ordenados y $dp[l][r]$ es el costo mínimo para cortar el tramo delimitado por l, r, la primera división puede ser cualquier k entre ambos:

$$dp[l][r] = \min_{l<k<r} \{dp[l][k] + dp[k][r] + (a_r - a_l)\}.$$

Base: $dp[l][l+1] = 0$. Complejidad $\mathcal{O}(K^3)$ para K cortes; memoria $\mathcal{O}(K^2)$. El repositorio también ilustra optimización de Knuth para condiciones especiales de la recurrencia.

3.5.8 DP sobre árboles: vertex cover mínimo

Sea $dp[u][1]$ costo mínimo del subárbol si seleccionamos u; $dp[u][0]$ si no lo seleccionamos. Entonces:

$$dp[u][0] = \sum_{v \ hijo \ de \ u} dp[v][1], \quad dp[u][1] = 1 + \sum_{v} \min(dp[v][0], dp[v][1]).$$

La respuesta es $\min(dp[root][0], dp[root][1])$. Esta recurrencia es para árbol; en grafos generales el problema es difícil.

3.5.9 DP de conteo: sumar S con K términos no negativos

Sea $dp[k][s]$ la cantidad de formas de escribir s como suma de k enteros no negativos (el orden importa). Transición:

$$dp[k][s] = \sum_{x=0}^{s} dp[k-1][s-x].$$

Con sumas prefijo se optimiza a $O(KS)$.

3.5.10 DP de presupuesto / compras

En problemas de prendas con presupuesto B, procesa categoría por categoría. Mantén alcanzables los gastos posibles tras escoger exactamente una opción de cada categoría. Si al final existe gasto $b \leq B$, maximiza b; si no, reporta imposible.

3.5.11 DP de recolección por alturas (ACORN)

Estado típico: mejor cantidad de bellotas alcanzable desde altura h; transición entre seguir en el mismo árbol (subir una altura) o saltar f alturas a otro árbol. Se procesa de mayor a menor altura y se combina el máximo entre árboles.

3.5.12 TSP pequeño / Beepers

Con punto inicial y k ubicaciones que visitar, usa $dp[mask][last]$: costo mínimo tras visitar el subconjunto $mask$ y terminar en $last$. Complejidad $O(k^2 2^k)$; cerrar el recorrido sumando distancia de retorno al origen si el enunciado lo exige.

3.5.13 DP con bitmask / matching perfecto pequeño

Señal del problema: Emparejar todos los elementos por pares minimizando costo, con pocos elementos.
Herramienta: Estado máscara de elementos aún libres.

```
// n debe ser par y pequeño (por ejemplo <= 20)
int FULL=(1<<n)-1;
vector<double> memo(1<<n,-1.0);
function<double(int)> dp = [&](int mask)->double {
    if(mask==0) return 0.0;
    double &ans=memo[mask];
    if(ans>=0) return ans;
    int i=__builtin_ctz((unsigned)mask);
    int rem=mask^(1<<i);
    ans=1e100;
    for(int j=i+1;j<n;j++) if(rem>>j&1)
        ans=min(ans,dist[i][j]+dp(rem^(1<<j)));
    return ans;
};
```

Complejidad: $\mathcal{O}(2^N N)$ estados/transiciones en esta formulación; memoria $\mathcal{O}(2^N)$.

3.6 Reconocer el paradigma correcto

Señal del enunciado	Primera idea a evaluar
$N \leq 10$ y pide todas las formas	backtracking / permutaciones
$N \leq 20$ y subconjuntos	DP bitmask o enumeración de máscaras
$N \sim 10^5$, orden y consulta por rango	sorting, Fenwick, Segment Tree
Mínimo x factible y monotonicidad	binary search sobre respuesta
Función con un único mínimo/máximo	ternary search
Subestructura óptima repetida	DP
Ordenar objetos por una llave y escoger	greedy, con prueba
Mejor camino en grafo	BFS / Dijkstra / Bellman-Ford
Conectar todos al menor costo	MST

CAPÍTULO 4

Grafos

4.1 Representaciones

```cpp
int n,m; cin>>n>>m;
vector<vector<pair<int,long long>>> g(n);
for(int i=0;i<m;i++) {
    int u,v; long long w; cin>>u>>v>>w;
    g[u].push_back({v,w});
    // si no dirigido:
    g[v].push_back({u,w});
}
```

- Lista de adyacencia: $\mathcal{O}(V+E)$ memoria; ideal para grafos dispersos.
- Matriz de adyacencia: $\mathcal{O}(V^2)$ memoria; arista en $\mathcal{O}(1)$, útil en grafos densos o Floyd-Warshall.
- Lista de aristas: cómoda para Kruskal y Bellman-Ford.

4.2 BFS: recorrido y camino más corto sin pesos

Señal del problema: Todas las aristas cuestan 1 (o grafo no ponderado).
Herramienta: Breadth-First Search.

```cpp
vector<int> bfs(int s,const vector<vector<int>>& g) {
    int n=g.size(); vector<int> d(n,-1),par(n,-1);
    queue<int> q; q.push(s); d[s]=0;
    while(!q.empty()) {
        int u=q.front(); q.pop();
        for(int v:g[u]) if(d[v]==-1) {
            d[v]=d[u]+1; par[v]=u; q.push(v);
        }
    }
    return d;
}
```

Complejidad: $\mathcal{O}(V+E)$. Desde un origen, da distancias mínimas por cantidad de aristas.

Reconstrucción: desde t, seguir `par[t]` hasta -1 y revertir. Si $d[t]=-1$, no hay camino.

4.2.1 BFS para bipartición

Colorea el origen con 0 y cada vecino con color opuesto. Si una arista une dos vértices del mismo color, no es bipartito. Repite en cada componente. Un grafo es bipartito si y solo si no contiene ciclos impares.

4.3 DFS, componentes y flood fill

```cpp
void dfs(int u,const vector<vector<int>>& g,vector<int>& vis) {
    vis[u]=1;
    for(int v:g[u]) if(!vis[v]) dfs(v,g,vis);
}
int components(const vector<vector<int>>& g) {
    int n=g.size(), cc=0; vector<int> vis(n,0);
    for(int u=0;u<n;u++) if(!vis[u]) {
        ++cc; dfs(u,g,vis);
    }
    return cc;
}
```

Complejidad: $\mathcal{O}(V+E)$. En grillas, los vecinos se generan con arreglos dr, dc de 4 u 8 direcciones y límites del tablero.

Ojo. Para V muy grande, una DFS recursiva puede causar stack overflow. Usa una pila explícita o aumenta el margen solo si el entorno lo permite.

4.4 Topological Sort

Señal del problema: Dependencias dirigidas sin ciclos; orden válido de tareas.
Herramienta: Kahn (BFS de indegree) o DFS postorden.

```cpp
vector<int> topoSort(const vector<vector<int>>& g) {
    int n=g.size(); vector<int> indeg(n,0);
    for(int u=0;u<n;u++) for(int v:g[u]) indeg[v]++;
    queue<int> q;
    for(int i=0;i<n;i++) if(indeg[i]==0) q.push(i);
    vector<int> order;
    while(!q.empty()) {
        int u=q.front();q.pop(); order.push_back(u);
        for(int v:g[u]) if(--indeg[v]==0) q.push(v);
    }
    if((int)order.size()!=n) return {}; // hay ciclo
    return order;
}
```

Complejidad: $\mathcal{O}(V+E)$. Un orden vacío puede significar ciclo (distingue también el caso $V=0$).

4.5 Componentes fuertemente conexas (SCC)

Para grafo dirigido, una SCC es un conjunto donde cada vértice alcanza a todos los demás. Kosaraju: DFS para tiempos de salida, invertir aristas y procesar en orden decreciente de salida. Tarjan: una sola DFS con `disc`, `low` y pila.

```cpp
void kosaraju(const vector<vector<int>>& g,
              vector<vector<int>>& gr,
              vector<int>& comp) {
    int n=g.size(); vector<int> vis(n,0), order;
    function<void(int)> dfs1=[&](int u){
        vis[u]=1;
        for(int v:g[u]) if(!vis[v]) dfs1(v);
        order.push_back(u);
    };
    for(int i=0;i<n;i++) if(!vis[i]) dfs1(i);
    comp.assign(n,-1); int id=0;
    function<void(int)> dfs2=[&](int u){
        comp[u]=id;
        for(int v:gr[u]) if(comp[v]==-1) dfs2(v);
    };
    reverse(order.begin(),order.end());
    for(int u:order) if(comp[u]==-1) { dfs2(u); ++id; }
}
```

Complejidad: $\mathcal{O}(V+E)$ tiempo y memoria. Construye gr invirtiendo cada arista $u \to v$ como $v \to u$.

4.6 Puentes y puntos de articulación

Señal del problema: Grafo no dirigido: arista cuya eliminación aumenta componentes (puente) o vértice cuya eliminación aumenta componentes (articulación).
Herramienta: DFS low-link.

$$low[u] = \text{mín}\{tin[u], tin[w] \text{ de back-edges}, low[v] \text{ de hijos}\}.$$

Para árbol DFS $u \to v$: (u, v) es puente si $low[v] > tin[u]$; u es articulación si $low[v] \geq tin[u]$ para algún hijo (salvo raíz); la raíz es articulación si tiene más de un hijo DFS.

```
int timer=0;
vector<int> tin,low,isArt;
vector<pair<int,int>> bridges;
void dfsBridge(int u,int p,const vector<vector<int>>& g) {
    tin[u]=low[u]=timer++;
    int children=0;
    for(int v:g[u]) {
        if(v==p) continue; // para multigrafo conviene saltar por ID de arista
        if(tin[v]!=-1) low[u]=min(low[u],tin[v]);
        else {
            dfsBridge(v,u,g); ++children;
            low[u]=min(low[u],low[v]);
            if(low[v]>tin[u]) bridges.push_back({u,v});
            if(p!=-1 && low[v]>=tin[u]) isArt[u]=1;
        }
    }
    if(p==-1 && children>1) isArt[u]=1;
}
```

Ojo. La plantilla anterior asume que no hay aristas paralelas. En multigrafos, identifica cada arista con un ID y omite solo la arista padre, no cualquier arista hacia el padre.

4.7 Caminos mínimos desde una fuente

4.7.1 Dijkstra

Señal del problema: Pesos no negativos; una fuente; grafo disperso.
Herramienta: Min-heap de pares (distancia, vértice).

```
vector<long long> dijkstra(int s,
 const vector<vector<pair<int,long long>>>& g) {
    const long long INFLL=(1LL<<62);
    int n=g.size(); vector<long long> d(n,INFLL);
    priority_queue<pair<long long,int>,
      vector<pair<long long,int>>,greater<pair<long long,int>>> pq;
    d[s]=0; pq.push({0,s});
    while(!pq.empty()) {
        auto [du,u]=pq.top(); pq.pop();
        if(du!=d[u]) continue; // entrada vieja
        for(auto [v,w]:g[u]) if(d[v]>du+w) {
            d[v]=du+w; pq.push({d[v],v});
        }
    }
    return d;
}
```

Complejidad: $\mathcal{O}((V + E) \log V)$. No usar con aristas negativas. Evita sumar desde infinito.

4.7.2 Bellman-Ford

Señal del problema: Pesos negativos o detectar ciclos negativos alcanzables desde s.
Herramienta: Relajar todas las aristas $V - 1$ veces y realizar una pasada adicional.

```cpp
struct Edge { int u,v; long long w; };
vector<long long> bellmanFord(int n,int s,
 const vector<Edge>& e,bool& negCycle) {
    const long long INFLL=(1LL<<62);
    vector<long long> d(n,INFLL); d[s]=0;
    for(int it=0;it<n-1;it++) {
        bool changed=false;
        for(auto x:e) if(d[x.u]!=INFLL &&
            d[x.v]>d[x.u]+x.w) {
            d[x.v]=d[x.u]+x.w; changed=true;
        }
        if(!changed) break;
    }
    negCycle=false;
    for(auto x:e) if(d[x.u]!=INFLL &&
        d[x.v]>d[x.u]+x.w) negCycle=true;
    return d;
}
```

Complejidad: $\mathcal{O}(VE)$. Detecta ciclos negativos que sean alcanzables desde el origen considerado.

4.7.3 Floyd-Warshall: todos contra todos

```cpp
const long long INFLL=(1LL<<60);
vector<vector<long long>> d(n,vector<long long>(n,INFLL));
for(int i=0;i<n;i++) d[i][i]=0;
for(auto [u,v,w]:edges) d[u][v]=min(d[u][v],w);
for(int k=0;k<n;k++)
  for(int i=0;i<n;i++) if(d[i][k]<INFLL)
    for(int j=0;j<n;j++) if(d[k][j]<INFLL)
      d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
// ciclo negativo si existe k con d[k][k] < 0
```

Complejidad: $\mathcal{O}(V^3)$ tiempo y $\mathcal{O}(V^2)$ memoria; adecuado para V pequeño.

4.8 Árbol de expansión mínima (MST)

Señal del problema: Conectar todos los vértices no dirigidos minimizando el costo total.
Herramienta: Kruskal + DSU o Prim + heap.

4.8.1 Kruskal

```cpp
struct WEdge { int u,v; long long w; };
sort(e.begin(),e.end(),[](auto a,auto b){return a.w<b.w;});
DSU dsu(n); long long cost=0; int used=0;
for(auto x:e) if(dsu.unite(x.u,x.v)) {
    cost+=x.w; ++used;
}
bool connected=(used==n-1);
```

Complejidad: $\mathcal{O}(E \log E)$. Si `used < n-1`, el grafo no es conexo y el resultado es un bosque de expansión mínimo.

4.8.2 Prim

```cpp
long long prim(int s,const vector<vector<pair<int,long long>>>& g) {
    int n=g.size(); vector<int> used(n,0);
    priority_queue<pair<long long,int>,
      vector<pair<long long,int>>,greater<pair<long long,int>>> pq;
    pq.push({0,s}); long long cost=0; int taken=0;
    while(!pq.empty()) {
        auto [w,u]=pq.top(); pq.pop();
        if(used[u]) continue;
        used[u]=1; cost+=w; ++taken;
        for(auto [v,c]:g[u]) if(!used[v]) pq.push({c,v});
    }
    return taken==n ? cost : -1; // -1 como indicador solo si el costo puede ser negativo no es ideal
}
```

Ojo. Prim anterior calcula desde una componente y usa -1 solo como ejemplo de indicador. Si los pesos pueden ser negativos, devuelve también `taken` o un par (conexo,costo).

4.9 Euler: camino y circuito

Un camino Euler usa cada arista exactamente una vez. Hierholzer consume aristas y construye el recorrido al retroceder. Para grafo dirigido balanceado, todos los vértices con grado relevante deben estar en la componente adecuada; en no dirigido, hay 0 o 2 vértices de grado impar para circuito/camino.

```
vector<int> hierholzer(int s, vector<vector<int>>& g) {
    vector<int> it(g.size()), ans, st={s};
    while(!st.empty()) {
        int u=st.back();
        if(it[u]<(int)g[u].size()) {
            st.push_back(g[u][it[u]++]);
        } else {
            ans.push_back(u); st.pop_back();
        }
    }
    reverse(ans.begin(),ans.end());
    return ans;
}
```

Ojo. La implementación requiere manejar cada arista una sola vez. En no dirigidos, guarda IDs de arista o marca ambos sentidos como usados. Valida al final que el recorrido haya consumido exactamente E aristas.

4.10 Matching bipartito máximo

Señal del problema: Asignar elementos de lado izquierdo a lado derecho sin repetir, maximizando pares.
Herramienta: Kuhn / DFS de caminos aumentantes.

```
vector<vector<int>> g; // solo aristas L -> R
vector<int> matchR, seen;
bool aug(int u) {
    if(seen[u]) return false;
    seen[u]=1;
    for(int v:g[u])
        if(matchR[v]==-1 || aug(matchR[v])) {
            matchR[v]=u; return true;
        }
    return false;
}
int maxMatching(int L,int R) {
    matchR.assign(R,-1); int ans=0;
    for(int u=0;u<L;u++) {
        seen.assign(L,0);
        ans+=aug(u);
    }
    return ans;
}
```

Complejidad: Kuhn $\mathcal{O}(VE)$ en el peor caso. Para grafos grandes, considera Hopcroft-Karp $\mathcal{O}(E\sqrt{V})$.

4.11 Elección del algoritmo de grafos

Problema	Algoritmo	Restricción clave
Distancia sin pesos	BFS	$O(V+E)$
Pesos no negativos	Dijkstra	ningún peso negativo
Pesos negativos, una fuente	Bellman-Ford	$O(VE)$
Distancias todos los pares	Floyd-Warshall	V pequeño
Conectar todo barato	MST	no dirigido
Dependencias	Topological Sort	DAG
Componentes dirigidas	SCC	dirigido
Puentes/articulaciones	low-link	no dirigido
Asignación entre dos lados	Matching	bipartito
Usar todas las aristas	Hierholzer	condiciones de grados/conectividad

CAPÍTULO 5

Matemáticas para programación competitiva

5.1 Aritmética entera básica

5.1.1 GCD, LCM y Euclides extendido

```cpp
long long gcdll(long long a,long long b) {
    while(b) { a%=b; swap(a,b); }
    return a;
}
long long lcmll(long long a,long long b) {
    if(a==0 || b==0) return 0;
    return a/gcdll(a,b)*b; // puede desbordar si el resultado no cabe
}
```

El algoritmo de Euclides usa $\gcd(a,b) = \gcd(b, a \bmod b)$ y toma $\mathcal{O}(\log \min(a,b))$. La identidad de Bézout asegura que existen x, y tales que $ax + by = \gcd(a,b)$.

```cpp
long long extgcd(long long a,long long b,long long& x,long long& y) {
    if(b==0) { x=1; y=0; return a; }
    long long x1,y1;
    long long g=extgcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1;
    return g;
}
```

5.2 Aritmética modular

$$(a+b) \bmod M = ((a \bmod M) + (b \bmod M)) \bmod M,$$
$$(ab) \bmod M = ((a \bmod M)(b \bmod M)) \bmod M.$$

En C++, el resto de un negativo puede ser negativo. Normaliza con:

```cpp
long long mod(long long a,long long m) {
    return (a%m+m)%m; // m > 0
}
```

5.2.1 Exponenciación binaria

```cpp
long long modpow(long long a,long long e,long long m) {
    long long r=1%m; a=mod(a,m);
    while(e>0) {
        if(e&1) r=(__int128)r*a%m;
        a=(__int128)a*a%m;
        e>>=1;
    }
    return r;
```

```
}
```

Complejidad: $\mathcal{O}(\log e)$. El uso de `__int128` evita overflow intermedio para operandos de 64 bits y módulo de 64 bits positivo dentro del rango firmado; para módulo $< 2^{63}$ basta con cuidado.

5.2.2 Inverso modular

El inverso de a módulo m existe si y solo si $\gcd(a, m) = 1$. Con Euclides extendido, si $ax + my = 1$, entonces $a^{-1} \equiv x \pmod{m}$.

```
long long invmod(long long a,long long m) {
    long long x,y, g=extgcd(a,m,x,y);
    if(g!=1) return -1; // no existe
    return mod(x,m);
}
```

Si m es primo y $a \not\equiv 0 \pmod{m}$, Fermat: $a^{-1} \equiv a^{m-2} \pmod{m}$. No uses esta fórmula para módulo compuesto en general.

5.3 Primos y factorización

5.3.1 Criba de Eratóstenes

```
vector<int> sieve(int n) {
    vector<bool> isPrime(n+1,true);
    if(n>=0) isPrime[0]=false;
    if(n>=1) isPrime[1]=false;
    for(long long i=2;i*i<=n;i++) if(isPrime[i])
        for(long long j=i*i;j<=n;j+=i) isPrime[j]=false;
    vector<int> p;
    for(int i=2;i<=n;i++) if(isPrime[i]) p.push_back(i);
    return p;
}
```

Complejidad: $\mathcal{O}(N \log \log N)$ tiempo, $\mathcal{O}(N)$ memoria. Empezar a tachar en p^2 evita trabajo redundante.

5.3.2 Factorización por división de prueba

```
vector<pair<long long,int>> factorize(long long n) {
    vector<pair<long long,int>> f;
    for(long long p=2;p*p<=n;p+=(p==2?1:2)) if(n%p==0) {
        int e=0;
        do { n/=p; ++e; } while(n%p==0);
        f.push_back({p,e});
    }
    if(n>1) f.push_back({n,1});
    return f;
}
```

Complejidad: $\mathcal{O}(\sqrt{N})$ divisiones en el peor caso. Para muchos valores pequeños, preprocesa el menor factor primo (SPF).

5.3.3 Funciones de divisores

Si $n = \prod p_i^{a_i}$:

$$d(n) = \prod_i (a_i + 1), \qquad \sigma(n) = \prod_i \frac{p_i^{a_i+1} - 1}{p_i - 1},$$

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right).$$

$d(n)$ cuenta divisores positivos; $\sigma(n)$ suma divisores positivos; $\varphi(n)$ cuenta enteros en $[1, n]$ coprimos con n.

```cpp
long long phi(long long n) {
    long long ans=n;
    for(long long p=2;p*p<=n;p++) if(n%p==0) {
        ans-=ans/p;
        while(n%p==0) n/=p;
    }
    if(n>1) ans-=ans/n;
    return ans;
}
```

5.3.4 Legendre: exponente de primo en $n!$

$$v_p(n!) = \left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \cdots$$

Para determinar si m divide a $n!$, factoriza $m = \prod p_i^{e_i}$ y comprueba $v_{p_i}(n!) \geq e_i$ para cada primo. Esto es mucho más seguro que calcular el factorial.

5.4 Combinatoria

5.4.1 Fórmulas esenciales

$$n! = 1 \cdot 2 \cdots n, \quad P(n,k) = \frac{n!}{(n-k)!}, \quad \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

$$\binom{n}{k} = \binom{n}{n-k}, \quad \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

Identidades útiles:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n, \qquad \sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1}.$$

Inclusión-exclusión: para conjuntos $A_1, \ldots, A_n$:

$$|\cup_i A_i| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \cdots + (-1)^{n+1} |\cap_i A_i|.$$

Se usa para contar objetos que cumplen "al menos una" condición restando intersecciones. **Estrellas y barras:** soluciones enteras no negativas de $x_1 + \cdots + x_k = n$:

$$\binom{n+k-1}{k-1}.$$

Soluciones positivas: $\binom{n-1}{k-1}$. **Catalan:** $C_n = \frac{1}{n+1}\binom{2n}{n}$; cuenta paréntesis balanceados, árboles binarios completos ordenados, triangulaciones de polígono convexo y otros objetos equivalentes.

5.4.2 Factoriales e inversos precomputados (módulo primo)

```cpp
const long long MOD=1'000'000'007;
long long modpow(long long a,long long e,long long m);
vector<long long> fact, ifact;
void initComb(int n) {
    fact.resize(n+1); ifact.resize(n+1);
    fact[0]=1;
    for(int i=1;i<=n;i++) fact[i]=fact[i-1]*i%MOD;
    ifact[n]=modpow(fact[n],MOD-2,MOD);
    for(int i=n;i>=1;i--) ifact[i-1]=ifact[i]*i%MOD;
}
long long C(int n,int k) {
    if(k<0 || k>n) return 0;
    return fact[n]*ifact[k]%MOD*ifact[n-k]%MOD;
}
```

Complejidad: Preprocesamiento $\mathcal{O}(N + \log MOD)$; cada combinación $\mathcal{O}(1)$. Requiere $n < MOD$ y módulo primo para el método mostrado.

5.4.3 Catalan y Fibonacci

$$C_0 = 1, \quad C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}, \quad C_n = \frac{1}{n+1}\binom{2n}{n}.$$

$$F_0 = 0,\ F_1 = 1,\ F_n = F_{n-1} + F_{n-2}.$$

Fast doubling calcula Fibonacci en $\mathcal{O}(\log n)$:

```
pair<long long,long long> fib(long long n,long long mod) {
    assert(mod>0);
    if(n==0) return {0,1%mod};
    auto [a,b]=fib(n/2,mod);
    long long c=(long long)((__int128)a*((2*(__int128)b-a+mod)%mod)%mod);
    long long d=(long long)(((__int128)a*a+(__int128)b*b)%mod);
    long long e=(long long)(((__int128)c+d)%mod);
    if(n&1) return {d,e};
    return {c,d};
}
```

5.5 Teorema chino del resto (CRT)

Para módulos coprimos m_i, el sistema $x \equiv r_i \pmod{m_i}$ tiene solución única módulo $M = \prod_i m_i$. Para dos congruencias coprimas se puede combinar con inverso modular.

```
// Combina x = a (mod m), x = b (mod n), gcd(m,n)=1.
long long mod128(__int128 x,long long m) {
    x%=m; if(x<0) x+=m;
    return (long long)x;
}
long long crt2(long long a,long long m,long long b,long long n) {
    a=mod(a,m); b=mod(b,n);
    long long inv=invmod(m,n);
    long long t=mod128((__int128)(b-a)*inv,n);
    return (long long)((__int128)a+(__int128)m*t); // exige m*n <= LLONG_MAX
}
```

Ojo. Los módulos deben ser coprimos en esta versión y mn debe caber en el tipo de retorno. Si no son coprimos, hay solución si y solo si $a \equiv b \pmod{\gcd(m,n)}$; combina usando Euclides extendido.

5.6 Otros patrones matemáticos del repositorio

- **Período modular:** algunas sucesiones módulo m son periódicas; detecta el período del estado (por ejemplo, pares consecutivos de Fibonacci) y reduce el índice.
- **Fracciones:** simplifica a/b dividiendo numerador y denominador por $\gcd(|a|,|b|)$; normaliza el signo para que el denominador sea positivo.
- **Cambio de base y restos:** usa división sucesiva para convertir, cuidando cero y números negativos.
- **Conteo combinatorio con restricciones:** identifica si son permutaciones, combinaciones, ordenamientos con repetición, inclusión-exclusión o DP.

CAPÍTULO 6

Procesamiento de cadenas

6.1 Coincidencia de patrones

Sea texto T de longitud n y patrón P de longitud m.

6.1.1 Naive matching

Compara el patrón en cada desplazamiento: $\mathcal{O}(nm)$. Sirve para cadenas pequeñas y validaciones.

6.1.2 KMP: Knuth-Morris-Pratt

La función prefijo $\pi[i]$ es la longitud del mayor prefijo propio de $P[0..i]$ que también es sufijo. Permite saltar comparaciones ya conocidas.

```
vector<int> prefixFunction(const string& s) {
    int n=s.size(); vector<int> pi(n);
    for(int i=1;i<n;i++) {
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) ++j;
        pi[i]=j;
    }
    return pi;
}
vector<int> kmp(const string& t,const string& p) {
    vector<int> ans;
    if(p.empty()) return ans;
    auto pi=prefixFunction(p); int j=0;
    for(int i=0;i<(int)t.size();i++) {
        while(j>0 && t[i]!=p[j]) j=pi[j-1];
        if(t[i]==p[j]) ++j;
        if(j==(int)p.size()) {
            ans.push_back(i-j+1); j=pi[j-1];
        }
    }
    return ans;
}
```

Complejidad: Preproceso $\mathcal{O}(m)$ y búsqueda $\mathcal{O}(n)$; encuentra ocurrencias solapadas.

6.1.3 Rabin-Karp / rolling hash

Asigna valores a caracteres y calcula hash de cada ventana; compara hashes para filtrar candidatos. La probabilidad de colisión se reduce usando dos módulos y dos bases, pero no desaparece matemáticamente. Para seguridad exacta, verifica la subcadena cuando hay hash igual.

```cpp
struct RollingHash {
    static const long long M1=1'000'000'007, M2=1'000'000'009;
    static const long long B=911382323; // usar base menor que los módulos
    vector<long long> h1,h2,p1,p2;
    RollingHash(const string& s) {
        int n=s.size(); h1.assign(n+1,0); h2.assign(n+1,0);
        p1.assign(n+1,1); p2.assign(n+1,1);
        for(int i=0;i<n;i++) {
            int x=(unsigned char)s[i]+1;
            h1[i+1]=(h1[i]*B+x)%M1;
            h2[i+1]=(h2[i]*B+x)%M2;
            p1[i+1]=p1[i]*B%M1; p2[i+1]=p2[i]*B%M2;
        }
    }
    pair<long long,long long> get(int l,int r) { // [l,r)
        long long a=(h1[r]-h1[l]*p1[r-l]%M1+M1)%M1;
        long long b=(h2[r]-h2[l]*p2[r-l]%M2+M2)%M2;
        return {a,b};
    }
};
```

Complejidad: Preproceso $\mathcal{O}(n)$; hash de subcadena $\mathcal{O}(1)$. Para el doble hash mostrado, revisa que los productos intermedios caben en 64 bits; con estos módulos y base elegida sí.

6.2 Trie (árbol de prefijos)

Señal del problema: Diccionario de palabras, consultas por prefijo, autocompletado.
Herramienta: Árbol de 26 hijos (alfabeto mayúsculo) o mapa para alfabeto general.

```cpp
struct Trie {
    struct Node { int nxt[26]; bool end;
        Node(): end(false) { fill(nxt,nxt+26,-1); }
    };
    vector<Node> t{1};
    void insert(const string& s) {
        int u=0;
        for(char c:s) {
            int x=c-'A';
            if(t[u].nxt[x]==-1) {
                t[u].nxt[x]=t.size(); t.emplace_back();
            }
            u=t[u].nxt[x];
        }
        t[u].end=true;
    }
    bool search(const string& s) const {
        int u=0;
        for(char c:s) {
            int x=c-'A'; if(t[u].nxt[x]==-1) return false;
            u=t[u].nxt[x];
        }
        return t[u].end;
    }
};
```

Complejidad: Insertar/buscar palabra de longitud L: $\mathcal{O}(L)$. Ajusta el alfabeto: esta plantilla solo admite A-Z.

6.3 Alineamiento y distancia de edición

Needleman-Wunsch alinea dos secuencias maximizando puntuación de match/mismatch e inserción/borrado. La distancia de Levenshtein minimiza inserciones, borrados y sustituciones.

```cpp
int editDistance(string a,string b) {
    int n=a.size(),m=b.size();
    vector<vector<int>> dp(n+1,vector<int>(m+1));
    for(int i=0;i<=n;i++) dp[i][0]=i;
    for(int j=0;j<=m;j++) dp[0][j]=j;
    for(int i=1;i<=n;i++) for(int j=1;j<=m;j++) {
        int cost=(a[i-1]==b[j-1]?0:1);
        dp[i][j]=min({dp[i-1][j]+1,dp[i][j-1]+1,
                      dp[i-1][j-1]+cost});
```

```
    }
    return dp[n][m];
}
```

Complejidad: $\mathcal{O}(nm)$ tiempo y memoria; se puede reducir a $\mathcal{O}(\text{mín}(n, m))$ memoria con dos filas.

6.4 Suffix Array y LCP

El suffix array ordena los sufijos $T[i..n-1]$ lexicográficamente. $LCP[k]$ guarda el largo del prefijo común entre los sufijos en posiciones k y $k-1$ del array ordenado.

```
struct SuffixArray {
    string s; vector<int> sa,rankv,lcp;
    SuffixArray(string str):s(str) { build(); }
    void build() {
        int n=s.size(); sa.resize(n); iota(sa.begin(),sa.end(),0);
        rankv.resize(n);
        for(int i=0;i<n;i++) rankv[i]=(unsigned char)s[i];
        vector<int> nr(n);
        for(int k=1;k<n;k*=2) {
            auto key=[&](int i){
                return pair<int,int>{rankv[i], i+k<n?rankv[i+k]:-1};
            };
            sort(sa.begin(),sa.end(),[&](int a,int b){return key(a)<key(b);});
            nr[sa[0]]=0;
            for(int i=1;i<n;i++)
                nr[sa[i]]=nr[sa[i-1]]+(key(sa[i-1])!=key(sa[i]));
            rankv.swap(nr);
            if(rankv[sa.back()]==n-1) break;
        }
        // Kasai: LCP[i] = LCP(sa[i], sa[i-1]); LCP[0]=0
        vector<int> pos(n);
        for(int i=0;i<n;i++) pos[sa[i]]=i;
        lcp.assign(n,0); int h=0;
        for(int i=0;i<n;i++) {
            int r=pos[i]; if(r==0) continue;
            int j=sa[r-1];
            while(i+h<n && j+h<n && s[i+h]==s[j+h]) ++h;
            lcp[r]=h; if(h) --h;
        }
    }
};
```

Complejidad: Esta implementación usa sort de pares en cada ronda: $\mathcal{O}(n \log^2 n)$; Kasai $\mathcal{O}(n)$. El repositorio incluye una variante con counting/radix sort que alcanza $\mathcal{O}(n \log n)$.

Aplicaciones: subcadena repetida más larga = máximo de LCP; ocurrencias de patrón = dos búsquedas binarias comparando el patrón con sufijos, luego obtener el rango de sufijos coincidentes; subcadena común más larga entre dos textos = máximo LCP de sufijos adyacentes de textos distintos tras concatenar con separador único.

CAPÍTULO 7

Geometría computacional

7.1 Puntos y vectores

```
const double EPS=1e-9, PI=acos(-1.0);
struct Point { double x,y;
    Point(double x=0,double y=0):x(x),y(y){}
};
struct Vec { double x,y; };
Vec toVec(Point a,Point b){return {b.x-a.x,b.y-a.y};}
double dot(Vec a,Vec b){return a.x*b.x+a.y*b.y;}
double cross(Vec a,Vec b){return a.x*b.y-a.y*b.x;}
double norm2(Vec a){return dot(a,a);}
double dist(Point a,Point b){return hypot(a.x-b.x,a.y-b.y);}
Point translate(Point p,Vec v){return {p.x+v.x,p.y+v.y};}
```

7.2 Orientación y área

Para puntos A, B, C, el producto cruzado $(B - A) \times (C - A)$:

- > 0: giro antihorario (C a la izquierda de AB).
- < 0: giro horario.
- $= 0$: colineales.

Para coordenadas enteras, usa `__int128` en el producto cruzado si los límites pueden ser grandes.

$$\text{área triángulo} = \frac{1}{2}|(B - A) \times (C - A)|.$$

7.2.1 Fórmula del zapatero (shoelace)

Para polígono ordenado con $P_n = P_0$:

$$A = \frac{1}{2}\left|\sum_{i=0}^{n-1} x_i y_{i+1} - x_{i+1} y_i\right|.$$

Perímetro: $\sum_i \text{dist}(P_i, P_{i+1})$.

7.3 Rectas

Representa recta como $ax + by + c = 0$. Para dos puntos distintos P, Q:

$$a = y_P - y_Q, \quad b = x_Q - x_P, \quad c = -(ax_P + by_P).$$

Dos rectas son paralelas si $a_1 b_2 - a_2 b_1 = 0$ (con tolerancia para double). Intersección resolviendo sistema 2×2; si determinante ≈ 0, no hay intersección única.

```
bool lineIntersection(double a1,double b1,double c1,
                      double a2,double b2,double c2,Point& p) {
    double det=a1*b2-a2*b1;
    if(fabs(det)<EPS) return false;
    p.x=(b1*c2-b2*c1)/det;
    p.y=(c1*a2-c2*a1)/det;
    return true;
}
```

7.4 Proyección y distancia punto-segmento

Para segmento AB y punto P, proyecta AP sobre AB con parámetro

$$t = \frac{(P-A)\cdot(B-A)}{\|B-A\|^2}.$$

Recorta t al intervalo $[0,1]$; el punto más cercano es $A + t(B-A)$. Si $A = B$, la distancia es $|PA|$.

```
Point closestOnSegment(Point p,Point a,Point b) {
    Vec ab=toVec(a,b), ap=toVec(a,p);
    double den=norm2(ab);
    if(den<EPS*EPS) return a;
    double t=dot(ap,ab)/den;
    t=max(0.0,min(1.0,t));
    return translate(a,{ab.x*t,ab.y*t});
}
```

7.5 Segmentos

Orientaciones + comprobación de colinealidad resuelven intersección de segmentos. Para enteros, evita EPS y usa signo exacto del cross con tipo suficientemente ancho. Considera por separado extremos colineales y segmentos degenerados.

7.6 Círculos

$$d = 2r, \quad C = 2\pi r, \quad A = \pi r^2.$$

Para ángulo central θ en radianes: arco $r\theta$, sector $\frac{1}{2}r^2\theta$, cuerda $2r\sin(\theta/2)$. Si θ está en grados, conviértelo por $\theta_{rad} = \theta\pi/180$. **Punto dentro/en/fuera:** comparar $(x-x_c)^2 + (y-y_c)^2$ con r^2; usa enteros si los valores caben.

Distancia de punto a recta $ax + by + c = 0$:

$$d(P,l) = \frac{|ax_P + by_P + c|}{\sqrt{a^2+b^2}}.$$

El reflejo de un punto respecto de una recta se obtiene proyectando el punto sobre la recta y duplicando el desplazamiento hacia esa proyección.

7.7 Triángulos

$$s = \frac{a+b+c}{2}, \qquad A = \sqrt{s(s-a)(s-b)(s-c)} \quad \text{(Herón)}.$$

$$r_{in} = \frac{A}{s}, \qquad R_{circ} = \frac{abc}{4A}.$$

Ley de cosenos: $c^2 = a^2 + b^2 - 2ab\cos C$. Área por dos lados y ángulo incluido: $A = \frac{1}{2}ab\sin C$.

Ojo. Antes de usar raíces, valida que los lados formen un triángulo: cada lado menor que la suma de los otros dos. Redondea el argumento pequeño negativo a cero solo si se debe a error de precisión, no para ocultar datos inválidos.

7.8 Polígonos

7.8.1 Punto dentro/fuera

Métodos habituales: ray casting (contar cruces de un rayo) o suma de ángulos. Primero comprueba si el punto está sobre alguna arista; el enunciado puede pedir clasificar interior/borde/exterior.

7.8.2 Convexidad

En un polígono simple ordenado, calcula el signo de los productos cruzados de triples consecutivos. Si hay signos positivos y negativos, es cóncavo; ignora ceros si se permiten vértices colineales.

7.8.3 Corte de polígono

Para cortar por una recta orientada $A \to B$, conserva vértices del lado deseado y agrega las intersecciones de aristas que cruzan la recta. El orden de A, B define qué lado es positivo.

7.9 Errores geométricos frecuentes

- No comparar doubles con `==`; usa tolerancia cuando los límites lo permitan.
- No mezclar grados y radianes.
- Una recta vertical no tiene pendiente finita: usa $ax + by + c = 0$.
- En productos cruzados de coordenadas enteras grandes, usa `__int128`.
- Define si los extremos de segmentos cuentan como intersección.
- El orden de los vértices del polígono debe ser cíclico; no ordenar puntos de un polígono antes de calcular su área.

CAPÍTULO 8

Búsqueda avanzada y flujo máximo

8.1 Búsqueda en espacio de estados

Un estado describe completamente una configuración; una transición es una operación permitida; el objetivo es alcanzar un estado final minimizando pasos/costo o contar estados posibles. Para hacer BFS/DP, codifica cada estado en una clave compacta y evita procesarlo dos veces.

8.1.1 BFS de estados

```
queue<State> q;
unordered_map<State,int,Hash> dist;
q.push(start); dist[start]=0;
while(!q.empty()) {
    State u=q.front(); q.pop();
    if(u==goal) break;
    for(State v: nextStates(u)) if(!dist.count(v)) {
        dist[v]=dist[u]+1; q.push(v);
    }
}
```

8.1.2 Bidirectional BFS / Meet in the Middle

Si las transiciones son reversibles (o puedes generar predecesores), expande desde inicio y meta. Guarda distancias de ambos lados y detente cuando se encuentren. Para factor de ramificación b y profundidad d, puede reducir visitas de $O(b^d)$ a aproximadamente $O(b^{d/2})$ por lado. No siempre es mejor: cada estado consume memoria y debes generar transiciones inversas correctas.

8.2 IDA*: Iterative Deepening A*

Señal del problema: Estado enorme, solución de profundidad/costo pequeña, heurística admisible.
Herramienta: DFS con límite $g + h$, aumentando el límite de forma iterativa.

```
// h(s) nunca debe sobreestimar el costo restante.
bool dfs(State& s,int g,int bound) {
    int h=heuristic(s);
    if(g+h>bound) return false;
    if(isGoal(s)) return true;
    for(auto move: legalMoves(s)) {
        apply(s,move);
        if(dfs(s,g+cost(move),bound)) return true;
        undo(s,move);
    }
    return false;
}
int idaStar(State start) {
    int bound=heuristic(start);
    while(bound<=MAX_BOUND) {
        State cur=start;
        if(dfs(cur,0,bound)) return bound;
        ++bound; // mejor aumentar al menor f que excedió, si se implementa
```

```
    }
    return -1;
}
```

Heurística Manhattan en 15-puzzle: para cada ficha, suma $|r - r^*| + |c - c^*|$ respecto de su posición objetivo; es admisible si cada movimiento cuesta 1. Evita movimientos inversos inmediatos y deshace cada movimiento al retroceder. En una implementación eficiente, el siguiente límite se actualiza al menor $g + h$ que excedió el actual (no necesariamente $+1$).

8.3 DP y backtracking con máscaras

Conjunto independiente máximo: seleccionar vértices sin aristas entre sí. En grafos pequeños, representa vecindad con máscara y explora candidatos; el repositorio cuenta conjuntos independientes/maximales. **Reinas con obstáculos:** mantener columnas y diagonales prohibidas en máscaras; elegir un bit disponible por fila. **Partición geométrica / chocolate:** un estado puede ser (dimensiones, subconjunto de piezas), y una transición separa el subconjunto en dos partes con áreas compatibles. **Asignación de parejas:** DP sobre máscara (capítulo 3).

Ojo. La complejidad exponencial se controla con N pequeño, bitmasks, simetría, memoización y poda. Antes de codificar, estima el número de estados (por ejemplo 2^N o $N!$).

8.4 Flujo máximo

Señal del problema: Enviar la mayor cantidad desde fuente s hasta sumidero t respetando capacidades.
Herramienta: Red residual + caminos aumentantes.

La red residual mantiene capacidad disponible hacia adelante y permite cancelar flujo previo en sentido inverso. Teorema max-flow min-cut: valor del flujo máximo = capacidad del corte mínimo.

8.4.1 Dinic

```
struct Dinic {
    struct Edge { int to, rev; long long cap; };
    int n; vector<vector<Edge>> g;
    vector<int> level,it;
    Dinic(int n):n(n),g(n),level(n),it(n){}
    void addEdge(int u,int v,long long c) {
        Edge a={v,(int)g[v].size(),c};
        Edge b={u,(int)g[u].size(),0};
        g[u].push_back(a); g[v].push_back(b);
    }
    bool bfs(int s,int t) {
        fill(level.begin(),level.end(),-1);
        queue<int> q; q.push(s); level[s]=0;
        while(!q.empty()) {
            int u=q.front();q.pop();
            for(auto &e:g[u]) if(e.cap>0 && level[e.to]<0) {
                level[e.to]=level[u]+1; q.push(e.to);
            }
        }
        return level[t]>=0;
    }
    long long dfs(int u,int t,long long f) {
        if(u==t || f==0) return f;
        for(int &i=it[u];i<(int)g[u].size();i++) {
            Edge &e=g[u][i];
            if(e.cap<=0 || level[e.to]!=level[u]+1) continue;
            long long got=dfs(e.to,t,min(f,e.cap));
            if(got) {
                e.cap-=got; g[e.to][e.rev].cap+=got;
                return got;
            }
        }
        return 0;
    }
    long long maxflow(int s,int t) {
        long long flow=0;
        while(bfs(s,t)) {
            fill(it.begin(),it.end(),0);
```

```
            while(long long f=dfs(s,t,(1LL<<62))) flow+=f;
        }
        return flow;
    }
};
```

Complejidad: Dinic general: $O(V^2E)$; para redes bipartitas unitarias se obtiene mejor cota. Edmonds-Karp usa BFS para cada camino aumentante y cuesta $O(VE^2)$.

Usos típicos: asignación, emparejamiento (reformulado como flujo), caminos disjuntos, capacidad de red, circulación con demandas (requiere super fuente/sumidero). Para arista no dirigida, modela dos aristas dirigidas con capacidad c cada una, según el problema.

8.5 Corte mínimo

Después de calcular máximo flujo, ejecuta BFS/DFS en la red residual desde s usando aristas residuales de capacidad positiva. Los vértices alcanzables forman el lado fuente del corte mínimo; las aristas originales que cruzan hacia no alcanzables pertenecen a un corte mínimo.

CAPÍTULO 9

Temas avanzados del repositorio

9.1 LCA: Lowest Common Ancestor

El LCA de u, v es el ancestro común más profundo. Se consulta en árboles enraizados.

9.1.1 Binary lifting

Preprocesa $up[k][v]$: ancestro de v a distancia 2^k. También guarda profundidad. Para consultar, eleva primero el nodo más profundo y luego eleva ambos desde la potencia mayor a la menor.

```
struct LCA {
    int n,LOG; vector<int> depth;
    vector<vector<int>> up,g;
    LCA(vector<vector<int>> G,int root=0):g(G) {
        n=g.size(); LOG=1;
        while((1<<LOG)<=n) ++LOG;
        up.assign(LOG,vector<int>(n,root)); depth.assign(n,0);
        function<void(int,int)> dfs=[&](int u,int p){
            up[0][u]=p;
            for(int v:g[u]) if(v!=p) {
                depth[v]=depth[u]+1; dfs(v,u);
            }
        };
        dfs(root,root);
        for(int k=1;k<LOG;k++) for(int v=0;v<n;v++)
            up[k][v]=up[k-1][up[k-1][v]];
    }
    int query(int a,int b) {
        if(depth[a]<depth[b]) swap(a,b);
        int d=depth[a]-depth[b];
        for(int k=0;k<LOG;k++) if(d>>k&1) a=up[k][a];
        if(a==b) return a;
        for(int k=LOG-1;k>=0;k--) if(up[k][a]!=up[k][b])
            a=up[k][a], b=up[k][b];
        return up[0][a];
    }
};
```

Complejidad: Preproceso $O(N \log N)$, consulta $O(\log N)$, memoria $O(N \log N)$.

Ojo. La plantilla asume un árbol conectado sin ciclos. Para bosque, recorre cada raíz y guarda componente. En árboles muy grandes, el DFS recursivo puede requerir versión iterativa.

9.1.2 Euler Tour + RMQ

Otra opción: DFS produce arreglo Euler de longitud $2N - 1$, arreglo de profundidades y primera aparición de cada nodo. LCA(u, v) es el nodo de profundidad mínima entre las primeras apariciones. Con Sparse Table, consulta $O(1)$ tras preproceso $O(N \log N)$; con Segment Tree, consulta $O(\log N)$.

9.2 Heavy-Light Decomposition (HLD)

Descompone un árbol en cadenas pesadas. En cada nodo, el hijo con mayor subárbol es pesado; las demás aristas inician cadenas ligeras. Cualquier camino de raíz a nodo cruza $O(\log N)$ cadenas. Se asigna una posición lineal a cada nodo y se guarda una Segment Tree/Fenwick sobre esas posiciones.

- Para consulta de camino $u \leftrightarrow v$: mientras las cabezas de cadena sean distintas, procesar el tramo de la cadena cuya cabeza está más profunda y saltar a su padre.
- Cuando ambos están en la misma cadena, procesar el intervalo entre sus posiciones.
- Si la operación es sobre aristas y no vértices, representa cada arista por el valor del hijo más profundo y excluye la posición del LCA.

Complejidad: Descomposición $O(N)$; consulta/actualización de camino con Segment Tree $O(\log^2 N)$.

9.3 Sparse Table avanzada

Además del RMQ sobre arreglos, la Sparse Table se usa sobre el arreglo Euler de LCA, máximos, mínimos y gcd. Para sumas no se puede usar la combinación de dos bloques solapados; usa prefijos o una estructura dinámica.

9.4 Eliminación Gaussiana

Resuelve $AX = b$ construyendo la matriz aumentada. Pivoteo parcial: en cada columna escoge la fila restante con mayor valor absoluto en esa columna para reducir error numérico; elimina debajo y luego sustituye hacia atrás.

```
// Matriz aumentada n x (n+1). Devuelve false si no hay
// solución única o el pivote es numéricamente cero.
bool gauss(vector<vector<double>>& a,vector<double>& x) {
    int n=a.size(); const double EPS=1e-10;
    for(int col=0;col<n;col++) {
        int piv=col;
        for(int r=col+1;r<n;r++)
            if(fabs(a[r][col])>fabs(a[piv][col])) piv=r;
        if(fabs(a[piv][col])<EPS) return false;
        swap(a[piv],a[col]);
        for(int r=col+1;r<n;r++) {
            double f=a[r][col]/a[col][col];
            for(int j=col;j<=n;j++) a[r][j]-=f*a[col][j];
        }
    }
    x.assign(n,0);
    for(int i=n-1;i>=0;i--) {
        double s=a[i][n];
        for(int j=i+1;j<n;j++) s-=a[i][j]*x[j];
        x[i]=s/a[i][i];
    }
    return true;
}
```

Complejidad: $O(N^3)$. La plantilla solo reporta solución única; para clasificar infinitas soluciones/inconsistencia, calcula rango de A y de la aumentada.

9.5 Miller-Rabin y Pollard Rho (factorización 64-bit)

Miller-Rabin es una prueba probabilística/determinista según las bases usadas. Para enteros unsigned de 64 bits, el conjunto de bases conocido $\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$ da una prueba determinista.

```
using u64=uint64_t; using u128=__uint128_t;
u64 mulmod(u64 a,u64 b,u64 m){return (u128)a*b%m;}
u64 powmod64(u64 a,u64 e,u64 m){
    u64 r=1;
    while(e){if(e&1)r=mulmod(r,a,m);a=mulmod(a,a,m);e>>=1;}
    return r;
}
bool isPrime64(u64 n){
    if(n<2) return false;
```

```
    for(u64 p:{2ULL,3ULL,5ULL,7ULL,11ULL,13ULL,17ULL,19ULL,23ULL,29ULL,31ULL,37ULL}) {
        if(n%p==0) return n==p;
    }
    u64 d=n-1,s=0; while(!(d&1)){d>>=1;++s;}
    for(u64 a:{2ULL,325ULL,9375ULL,28178ULL,450775ULL,9780504ULL,1795265022ULL}) {
        if(a%n==0) continue;
        u64 x=powmod64(a%n,d,n);
        if(x==1 || x==n-1) continue;
        bool comp=true;
        for(u64 r=1;r<s;r++) { x=mulmod(x,x,n); if(x==n-1){comp=false;break;} }
        if(comp) return false;
    }
    return true;
}
```

Pollard Rho: encuentra un factor no trivial de un compuesto usando una función pseudoaleatoria $f(x) = x^2 + c$ (mód n) y gcd de diferencias. Combínalo recursivamente con Miller-Rabin. Requiere cuidado con desbordamiento y aleatoriedad; útil cuando $\sqrt{n}$ ya es demasiado grande para trial division.

```
u64 pollard(u64 n){
    if(n%2==0) return 2;
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    while(true){
        u64 c=uniform_int_distribution<u64>(1,n-1)(rng);
        u64 x=uniform_int_distribution<u64>(0,n-1)(rng), y=x, d=1;
        auto f=[&](u64 v){return (u128(mulmod(v,v,n))+c)%n;};
        while(d==1){
            x=f(x); y=f(f(y));
            u64 diff=x>y?x-y:y-x; d=gcd(diff,n);
        }
        if(d!=n) return d;
    }
}
void factor64(u64 n,vector<u64>& f){
    if(n==1) return;
    if(isPrime64(n)){f.push_back(n);return;}
    u64 d=pollard(n); factor64(d,f); factor64(n/d,f);
}
```

Ojo. En la suma $(x^2 + c)$ mód n de `f`, evita overflow si n puede acercarse al máximo unsigned: usa suma en `u128` o normaliza con aritmética modular extendida. Pollard Rho es aleatorio; puede necesitar varios reinicios. La plantilla Miller-Rabin anterior supone compilador con `__uint128_t`.

9.6 Minimum-Cost Maximum-Flow (MCMF)

Busca primero el mayor flujo y, entre flujos máximos, el de menor costo. Cada arista tiene capacidad y costo; la residual inversa tiene costo negativo. El repositorio muestra una variante con SPFA en la residual. La técnica general es camino aumentante de costo mínimo repetido.

```
struct MCMF {
    struct E { int to, rev; long long cap, cost; };
    int n; vector<vector<E>> g;
    MCMF(int n):n(n),g(n){}
    void addEdge(int u,int v,long long cap,long long cost){
        E a={v,(int)g[v].size(),cap,cost};
        E b={u,(int)g[u].size(),0,-cost};
        g[u].push_back(a); g[v].push_back(b);
    }
    pair<long long,long long> run(int s,int t){
        const long long INFLL=(1LL<<62);
        long long flow=0,cost=0;
        while(true){
            vector<long long>d(n,INFLL);
            vector<int> pv(n,-1),pe(n,-1),inq(n,0);
            queue<int>q; q.push(s); d[s]=0; inq[s]=1;
            // SPFA: soporta costos residuales negativos;
            // no debe haber ciclo negativo alcanzable relevante.
            while(!q.empty()){
                int u=q.front();q.pop();inq[u]=0;
                for(int i=0;i<(int)g[u].size();i++){
                    E &e=g[u][i];
                    if(e.cap>0 && d[e.to]>d[u]+e.cost){
                        d[e.to]=d[u]+e.cost;pv[e.to]=u;pe[e.to]=i;
```

```
                        if(!inq[e.to])inq[e.to]=1,q.push(e.to);
                    }
                }
            }
            if(d[t]==INFLL) break;
            long long add=INFLL;
            for(int v=t;v!=s;v=pv[v]) add=min(add,g[pv[v]][pe[v]].cap);
            for(int v=t;v!=s;v=pv[v]){
                E &e=g[pv[v]][pe[v]]; e.cap-=add;
                g[v][e.rev].cap+=add;
            }
            flow+=add; cost+=add*d[t];
        }
        return {flow,cost};
    }
};
```

Complejidad: SPFA por aumento: peor caso alto; no es buena elección para entradas adversarias grandes. Para rendimiento robusto, usar Dijkstra con potenciales (y Bellman-Ford inicial si hay costos negativos).

9.7 Teorema chino del resto: aplicación astronómica

Los problemas de sincronización de períodos, como una órbita que se repite cada a días y otra cada b días, se expresan como congruencias. Si $\gcd(a, b) = 1$, existe un instante único módulo ab. Esto transforma una simulación de hasta ab pasos en una solución con CRT.

9.8 Resumen de algoritmos avanzados

Técnica	Qué resuelve	Nota
IDA*	búsqueda con heurística	h admisible
Bidirectional BFS	caminos cortos en espacio de estados	transiciones inversas
Dinic	flujo máximo	red residual
MCMF	flujo máximo de costo mínimo	costos residuales negativos
LCA	ancestro común	árbol enraizado
HLD	consultas de camino en árbol	Segment Tree sobre cadenas
Gaussian	sistema lineal	pivoteo/tolerancia
Pollard Rho	factorizar enteros grandes	aleatorio + Miller-Rabin

CAPÍTULO A

Mapa de cobertura del repositorio original

El repositorio stevenhalim/cpbook-code se indexó desde su árbol recursivo de la rama `master`: contiene nueve directorios de capítulo y 433 entradas en el árbol consultado. El README lo describe como el proyecto de código fuente libre de CP4, con C++/GNU++17, Java 11, Python 3 y OCaml. Esta guía consolida las técnicas en una sola versión C++17 y añade notas de seguridad/uso; no es una transcripción línea por línea de todos los programas del repositorio.

A.1 Correspondencia por capítulo

Repositorio	Archivos / familias	Contenido cubierto en este Teambook
`ch1`	`IO`, `scanf`, `basic_string`, `factorial`, `moscowdream`, `UVa11616`, `zip.py`, ejercicios `Ex_1.3.4.1`	Entrada por casos/centinela/EOF, lectura de líneas, cadenas, conversión/formato, aritmética y problemas Ad Hoc.
`ch2/lineards`	`array_algorithms`, `bit_manipulation`, `list`, `resizeable_array`	STL, vectores/listas, ordenamiento multi-criterio, búsqueda binaria, permutaciones y bitmasks.
`ch2/nonlineards`	`AVL`, `QuickSelect`, `map_set`, `pbds`, `priority_queue`, `unordered_map_unordered_set`	BST/AVL, selección k-ésima, mapas/conjuntos ordenados y hash, heaps y PBDS.
`ch2/ourown`	`fenwicktree_ds`, `segmenttree_ds`, `unionfind_ds`, `graph_ds`	Fenwick PURQ/RUPQ/RURQ, Segment Tree/RMQ/lazy, DSU y representaciones de grafos.
`ch3/cs`	`UVa00441`, `UVa00725`, `UVa00750`, `UVa11565`, `UVa11742`, `UVa12455`	Complete Search, combinaciones, permutaciones, restricciones y backtracking.
`ch3/dnc`	`UVa11935`, `tricktreat`	Divide and Conquer, búsqueda binaria/ternaria y optimización unimodal.
`ch3/greedy`	`ballotboxes_UVa12390`, `grass_UVa10382`, `loowater_UVa11292`	Greedy, cobertura de intervalos, asignación y búsqueda binaria de respuesta.
`ch3/dp`	`LIS`, `Max1DRangeSum`, `UVa00108`, `UVa00674`, `UVa10003` (+ Knuth), `UVa10130`, `UVa10943`, `UVa11450`, `beepers_UVa10496`	DP 1D/2D, Kadane, LIS, cambio de monedas, mochila, interval DP, optimización de Knuth, DP de máscaras.
`ch4`	`graph_ds`, `traversal/*`, `sssp/*`, `mst/*`, `floyd_warshall`, `hierholzer`, `mcbm`	BFS/DFS, componentes, ciclos, topological sort, SCC, puentes/articulaciones, caminos mínimos, MST, Euler y matching bipartito.
`ch5`	`primes`, `combinatorics`, `modInverse`, `factovisors_UVa10139`, `basicremains_UVa10551`, `UVa00350`, `UVa01230`, `UVa10229`, `UVa10235`, `UVa10814`	Primos, criba, factorización, funciones de divisores, phi, modularidad, Euclides extendido, inversos, CRT, combinatoria y sucesiones modulares.
`ch6`	`string_matching`, `string_alignment`, `Trie`, `sa_lcp`, `sa_lcp_slow`	Naive/KMP/Rabin-Karp, alineamiento/edición, Trie, suffix array, LCP, subcadena repetida y búsqueda de patrón.

Repositorio	Archivos / familias	Contenido cubierto en este Teambook
`ch7`	`points_lines`, `triangles`, `circles`, `polygon`, `UVa11817`	Puntos, vectores, rectas, intersecciones, círculos, triángulos, área, perímetro, convexidad y punto en polígono.
`ch8`	`maxflow`, `UVa01079`, `UVa01099`, `UVa01231`, `UVa01238`, `UVa10243`, `UVa10911`, `UVa11065`, `UVa11195`, `UVa11212`, `UVa11646`	Flujo máximo, búsqueda en estados, backtracking con máscaras, matching, DP de subconjuntos/árboles, meet-in-the-middle, IDA* y búsqueda numérica.
`ch9`	`GaussianElimination`, `HLD`, `LCA`, `Pollardsrho`, `SparseTable`, `mcmf`, `UVa10181`, `heliocentric`	Eliminación Gaussiana, descomposición HLD, LCA/RMQ, Pollard Rho, Sparse Table, flujo de costo mínimo, IDA* y CRT.

A.2 Archivos de apoyo y ejemplos de jueces

Los archivos `*_in.txt` del repositorio son entradas de demostración para ejecutar los programas localmente; no son algoritmos independientes. Los archivos `UVa*`, `Kattis*` y nombres de problemas sirven como casos de aplicación y regresión. Cuando un problema del juez exige un detalle propio (formato de entrada, unidades, desempate, etc.), ese detalle no debe confundirse con la plantilla general.

A.3 Qué se consolidó y qué se adaptó

- Se priorizó C++17/GNU++17, que es el estándar de los ejemplos C++ del repositorio.
- Se normalizaron nombres de funciones, tipos y convenciones de índices.
- Se usaron implementaciones canónicas legibles para las estructuras más consultadas (DSU, Fenwick, Segment Tree, Dijkstra, Dinic, etc.).
- Algunas variantes del repositorio se explican como idea y no como una copia exacta: PBDS es extensión de GNU, las soluciones UVa dependen de su enunciado, y algunas implementaciones avanzadas requieren adaptaciones de límites.
- Para geometría con coordenadas enteras grandes y aritmética de 64 bits se recomiendan tipos extendidos; para flotantes se indica la necesidad de tolerancia.

A.4 Fuentes y atribución

Fuente principal: CP4 Free Source Code Project, de Steven Halim, Felix Halim y Suhendry Effendy. Rama consultada: `master`. El propio README invita a adaptar y usar los códigos en concursos y proyectos, y solicita citar a sus autores. Este documento es una guía de estudio/consulta derivada y reescrita, no sustituye al repositorio ni al libro CP4.

Recordatorio final para el equipo

La mejor caja negra no es la que tiene más código: es la que permite elegir la herramienta correcta y saber cuándo NO usarla. Antes de entregar, valida la complejidad, los límites, los tipos numéricos, el formato y las precondiciones. Si un algoritmo no coincide con la estructura del problema, ninguna optimización de implementación lo va a salvar.